

AI · COMPUTER VISION · VIVA PREPARATION

Smart Surveillance System

Complete Technical & Viva-Voce Preparation Report

Stack: Python · OpenCV · YOLOv8 · Flask

Scope: Architecture · Models · ML Metrics · Flask

Smart Surveillance System

Complete Viva-Voce Preparation & Technical Report

Purpose: This document prepares you to confidently defend the project in a viva. It explains the **high-level architecture**, the **data flow**, the **machine-learning models** (with their efficiency, accuracy, and error characteristics), the **mathematics**, and the **Flask web framework** including every important file and what it does.

Project: Real-time AI surveillance platform (Python, OpenCV, YOLOv8, Flask) **Reading time:** ~45 minutes · **Audience:** examiner / evaluator / new developer

Table of Contents

1. One-Paragraph Project Summary
2. What Problem Does It Solve?
3. High-Level Architecture
4. End-to-End Data Flow
5. The Machine-Learning Models — Overview
6. Model Efficiency, Accuracy & Error Factors (ML Metrics)
7. Per-Feature Model Card Summary
8. Flask Web Framework — Full Explanation
9. Main Flask Files and What Each Does

- 10. [Real-Time Communication \(Socket.IO\)](#)
- 11. [Databases & Persistence](#)
- 12. [Performance Optimizations](#)
- 13. [Key Mathematics Cheat-Sheet](#)
- 14. [Likely Viva Questions & Model Answers](#)
- 15. [Honest Limitations \(and How to Defend Them\)](#)
- 16. [Glossary of Terms](#)

1. One-Paragraph Project Summary

The **Smart Surveillance System** is a real-time, modular computer-vision platform that takes a video stream (webcam / IP camera / file), **detects objects** with a deep neural network (YOLOv8), **tracks** them across frames, and runs a large suite of **higher-level analytics** — facial recognition, behavior/pose analysis, person re-identification, fall / loitering / abandoned-object detection, crowd density, fire & smoke, weapon, violence, PPE compliance, and license-plate recognition. Detected events become **standardized, throttled alerts** that are stored in **SQLite databases** and pushed live to a **Flask + Socket.IO web dashboard** that a security operator monitors in the browser.

2. What Problem Does It Solve?

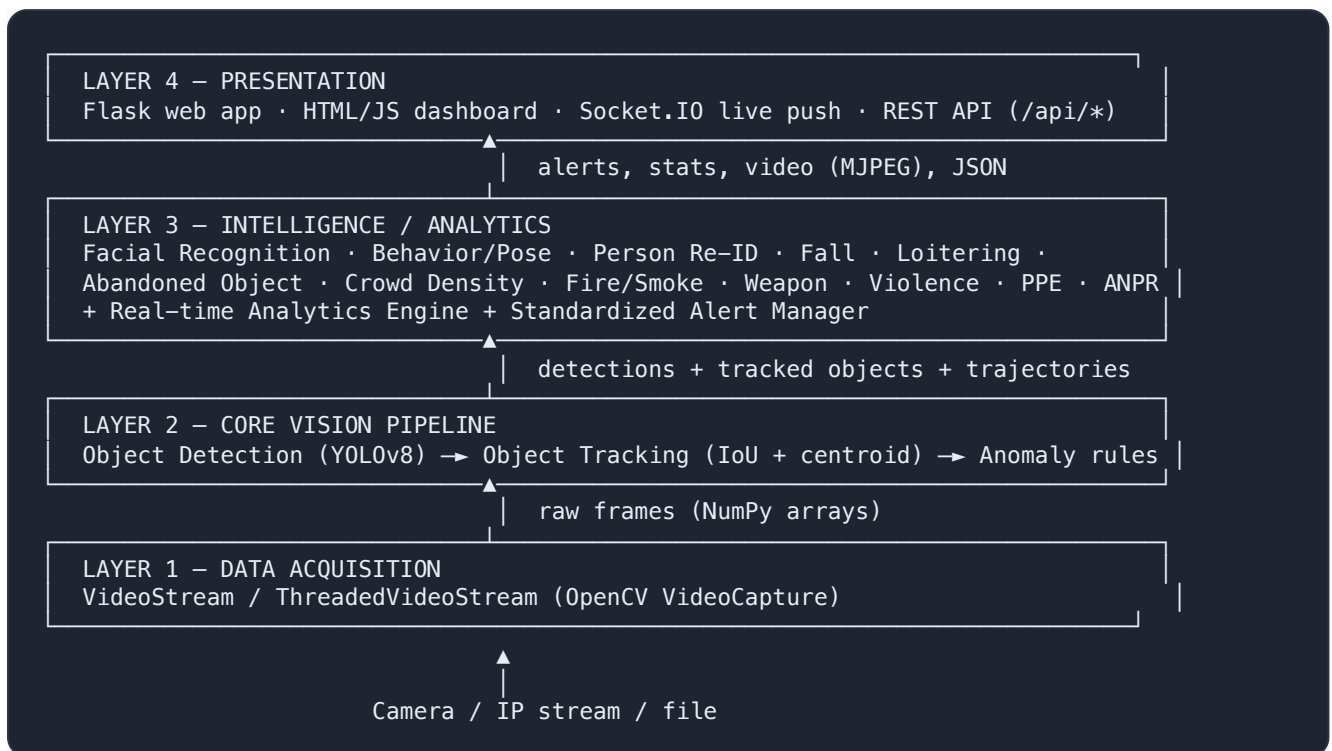
Traditional CCTV is **passive** — it records footage that a human must watch. This system makes surveillance **active and intelligent**:

Traditional CCTV	This System
Human watches all screens	AI watches and only alerts on events
Misses events due to fatigue	Never tires, monitors 24/7
Forensic (after the fact)	Real-time alerting
No analytics	Counts, identifies, classifies behavior
One capability	15+ integrated detection features

Application domains: access control, intrusion detection, elderly-care (falls), workplace safety (PPE), public-space safety (violence/weapons), transit security (abandoned luggage), traffic/ANPR.

3. High-Level Architecture

The system has **four logical layers**. Each layer is independent and communicates through simple Python data structures (lists of dictionaries) and a shared, lock-protected statistics object.



Cross-cutting services (used by all layers): - **Configuration** (`config/settings.py` , `config/model_config.py` , `.env` environment variables) - **Persistence** (one SQLite DB per subsystem) - **Logging** (timestamped log files in `logs/`)

Why this architecture is good (defendable design choices)

1. **Separation of concerns** — each detector is a self-contained class with `detect()` / `update()` and `draw_*()` methods. You can add or remove a feature without touching the rest.
2. **Graceful degradation** — optional dependencies (dlib, MediaPipe, EasyOCR) are wrapped in `try/except` . If a library is missing, that feature disables itself instead of crashing the app.
3. **Device portability** — automatic device selection **CUDA** → **MPS (Apple GPU)** → **CPU**.
4. **Thread isolation** — the dashboard runs in its own thread/process so video processing never blocks the UI, and vice-versa.

4. End-to-End Data Flow

For **every video frame**, the following happens (the "frame life-cycle"):

```

(1) CAPTURE          VideoStream.read_frame() → frame (NumPy H×W×3, BGR)
|
(2) DETECT           detector.detect(frame)
                    → list of {bbox:[x1,y1,x2,y2], class_id, class_name, confidence}
                    (YOLOv8 forward pass → confidence filter → NMS)
|
(3) TRACK            tracker.update(detections)
                    → {id: {centroid, bbox, class, trajectory[...]}}
                    (build cost matrix from IoU + centroid distance → greedy match)
|
(4) ANALYZE (parallel branches, all consume detections / tracks)
    — Facial recognition → identities (known/unknown + confidence)
    — Behavior analysis  → activity (walking/running/...) + suspicious flag
    — Anomaly detection  → unattended object / restricted-area violation
    — Person Re-ID       → cross-camera identity
    — Fall / Loitering / Abandoned / Crowd / Fire / Weapon / Violence / PPE / ANPR
|
(5) ALERT            StandardizedAlertManager.create_alert(type, msg, priority, data)
                    → THROTTLE (drop if same alert fired recently)
                    → DEDUPE (skip near-duplicate of recent event)
                    → PRIORITIZE (low / medium / high)
|
(6) PERSIST + PUSH   save to SQLite (alerts.db, ...) + socketio.emit('new_alert', ...)
|
(7) DISPLAY          MJPEG video feed + live stats + alert feed in the browser

```

Concrete example: an unknown person enters a restricted area

1. **Detect** → YOLO finds a `person` box (confidence 0.91).
2. **Track** → assigned ID 7, trajectory recorded.
3. **Face** → a face is detected but matches no known identity → "Unknown".
4. **Anomaly** → person's centroid is inside the restricted polygon (`pointPolygonTest ≥ 0`).
5. **Alert** → two alerts: `unknown_person` (HIGH) and `restricted_area_violation` (HIGH).
6. **Throttle** → if the same alert fired < 5 s ago it is suppressed (intrusion throttle window).
7. **Persist + push** → written to `alerts.db` , emitted to the browser instantly via Socket.IO.

5. The Machine-Learning Models — Overview

The system mixes **deep-learning models** (learned from data) with **classical / heuristic algorithms** (hand-designed rules). Knowing which is which is a common viva question.

5.1 Deep-learning models (learned)

Model	Type	Where used	Output
YOLOv8 (Ultralytics)	Single-stage CNN object detector	Object/weapon detection	Boxes + class + confidence
YuNet (ONNX)	CNN face detector	Facial recognition	Face boxes + 5 landmarks
SFace (ONNX)	CNN face recognizer	Facial recognition	128-D face embedding
dlib face_recognition	HOG/CNN + ResNet	Facial recognition (fallback)	128-D encoding
MediaPipe Pose	CNN landmark model	Behavior analysis	33 body landmarks
EasyOCR	CNN + CRNN/CTC	License-plate reading	Text + confidence

5.2 Classical / heuristic algorithms (no training)

Algorithm	Where used	Principle
IoU + centroid tracker	Object tracking	Geometric box overlap + distance
Point-in-polygon (ray casting)	Restricted-area / zones	Computational geometry
HSV color thresholding + morphology	Fire/smoke, PPE	Color segmentation
Aspect-ratio + velocity heuristics	Fall detection	Geometry + kinematics
Color histogram + Local Binary Pattern	Person Re-ID	Hand-crafted appearance features
Cosine similarity / Euclidean distance	Face & Re-ID matching	Vector similarity
Linear regression (least squares)	Analytics trends	Statistics

Important honesty note for the viva: The dashboard "feature detail" pages (marketing copy in `feature_data.py`) describe some features with *aspirational* models — e.g. "Deep SORT + Kalman" for tracking, "FaceNet" for faces, "CNN+LSTM" for violence. The **actual running code** uses a simpler greedy IoU tracker, YuNet/SFace (or dlib) for faces, and motion-statistics heuristics for violence. Be ready to state what is *implemented* vs. what is *described as a future upgrade*.

5.3 YOLOv8 in depth (the primary model)

YOLO = "You Only Look Once". It is a **single-stage** detector: one forward pass through a CNN predicts all boxes simultaneously (unlike two-stage detectors such as Faster R-CNN that first propose regions, then classify them). This is what makes it **real-time**.

Architecture: - **Backbone:** CSPDarknet with C2f modules — extracts hierarchical features. - **Neck:** PANet (Path Aggregation Network) — fuses features across scales (small→large objects). - **Head:** decoupled, **anchor-free** — predicts box geometry, objectness, and class scores at three scales (strides 8/16/32).

Final confidence per box:

```
score = P(object) · max_c P(class_c | object)
```

Boxes with `score ≥ conf_threshold` (0.4 here) are kept, then **Non-Maximum Suppression** removes duplicates using $\text{IoU} \geq 0.45$.

Model sizes (speed/accuracy trade-off): `n < s < m < l < x`. This project defaults to `yolov8s` (good balance). It is trained on **COCO** (80 classes, ~330k images).

6. Model Efficiency, Accuracy & Error Factors (ML Metrics)

This is the section most viva examiners focus on. Below are the **standard ML evaluation metrics**, what they mean, how they are computed in this project, and the **error factors**.

6.1 Detection metrics (the evaluator computes these — `evaluator.py`)

A detection is a **True Positive (TP)** only if the predicted class is correct **and** `IoU(prediction, ground-truth) ≥ threshold`. Otherwise it is a **False Positive (FP)**. A missed real object is a **False Negative (FN)**.

Metric	Formula	Meaning	"Good" value
Precision (P)	$TP / (TP + FP)$	Of the things I flagged, how many were real? (low FP)	> 0.9
Recall (R)	$TP / (TP + FN)$	Of the real things, how many did I catch? (low FN)	> 0.9
F1-score	$2PR / (P + R)$	Harmonic mean — single balanced score	> 0.9
mAP@0.5	mean over classes of AP at IoU 0.5	Standard detection accuracy	~0.9 (YOLOv8s on COCO ≈ 0.45 mAP)
mAP@0.5:0.95	AP averaged over IoU 0.5...0.95	Stricter COCO primary metric	higher = better

Average Precision (AP) is the **area under the Precision–Recall curve**:

$$AP = \int_0^1 P(R) \, dR \quad (\text{approximated by summing over PR points})$$

The Precision–Recall trade-off (key concept): - Lowering the confidence threshold → **higher recall, lower precision** (catch more, but more false alarms). The project deliberately does this for **weapon detection** (safety-critical: never miss). - Raising the threshold → **higher precision, lower recall** (fewer false alarms, but may miss some).

6.2 Speed / efficiency metrics

Metric	What it measures	In this project
FPS (frames/sec)	Throughput	Target 30 FPS; measured as $1/\Delta t$
Latency (ms/frame)	Delay per frame	preprocess + inference + postprocess time (from <code>results.speed</code>)
Frame-skip factor	Compute saved	Detection runs every 3rd frame (<code>FRAME_SKIP=3</code>) → ~3× speedup
Inference resolution <code>imgsz</code>	Accuracy vs cost	640/960/1280; cost grows ~quadratically
Model size	Memory & speed	<code>yolov8s</code> ≈ 11M params, small footprint

Efficiency levers the project actually uses: 1. **Frame skipping** — detect every Nth frame, reuse cached results between (3× fewer inferences). 2. **Threaded capture** — `ThreadedVideoStream`

decouples camera I/O from processing. 3. **Half precision (FP16)** + AMP on GPU. 4. **Auto device selection** — uses Apple MPS or CUDA when present. 5. **Class filtering** — only run/keep security-relevant classes to cut postprocessing noise. 6. **Adaptive `imgsz`** — drops 960→640 on CPU to keep FPS usable.

6.3 Error factors (sources of mistakes) — be ready to discuss

Error type	Cause	Mitigation in project
False Positives	Heuristic detectors (fire color = bright clothing; PPE color match)	Multi-frame verification (e.g., fire needs 60% of 5 frames); confidence gating
False Negatives	Small/distant/occluded objects	Higher <code>imgsz</code> ; lower confidence for weapons
ID switches	Greedy tracker loses identity through occlusion	<code>max_disappeared=30</code> tolerance; upgrade path = Deep SORT + Kalman
Class confusion	COCO has only 80 classes; unknown items mislabeled	Allow-list of relevant classes
Domain shift	Model trained on COCO, deployed on CCTV angle/lighting	CLAHE preprocessing option; custom fine-tuning via <code>trainer.py</code>
Threshold sensitivity	Fixed pixel thresholds vary with camera distance	Documented limitation; needs camera calibration/homography
Lighting / motion blur	Degrades faces & OCR	Face quality score = Laplacian-variance × brightness filters bad crops

6.4 Generalization concepts (overfitting/underfitting)

- **Overfitting** = model memorizes training data, fails on new data (high train accuracy, low validation). Counter-measures used in `trainer.py`: **data augmentation** (HSV jitter, flips, mosaic, scale/translate), **weight decay 5e-4**, **early stopping (patience 50)**, **dropout** option, and a held-out **validation split (20%)**.
- **Underfitting** = model too simple / undertrained (low train *and* validation accuracy). Counter: larger model (`m/l/x`), more epochs, higher learning rate warmup.
- **Train/Val/Test split**: training learns weights, validation tunes hyper-parameters & early stopping, test gives the final unbiased score (`mAP`).

6.5 Confusion matrix (classification view)

For per-class analysis the evaluator can produce a **confusion matrix** (`sklearn`), which cross-tabulates predicted vs. actual classes. The diagonal = correct; off-diagonal = confusions. From it you derive per-class precision/recall and spot which classes the model mixes up.

7. Per-Feature Model Card Summary

A quick "model card" per feature: what it uses, how it decides, and its main error mode.

Feature	Technique	Decision rule (core math)	Main error factor
Object Detection	YOLOv8 CNN	$\text{score} = P(\text{obj}) \cdot P(\text{cls})$, NMS IoU 0.45	small/occluded objects
Object Tracking	IoU + centroid, greedy	cost = $(1 - \text{IoU}) \cdot 100$ or Euclidean dist	occlusion → ID switch
Anomaly (unattended)	Trajectory rule	stationary if move < 5px for 30 frames	crowded scenes
Anomaly (restricted)	Point-in-polygon	$\text{pointPolygonTest} \geq 0$	calibration of zone
Facial Recognition	YuNet+SFace / dlib	cosine ≥ 0.363 (or dist ≤ 0.6)	pose/lighting/blur
Behavior/Pose	MediaPipe + rules	speed thresholds (5/20 px/frame)	speed not metric
Person Re-ID	Color hist + LBP	cosine sim > 0.7	weak hand-crafted features
Fall Detection	Aspect ratio + velocity	$\text{ar} > 1$ or (height↓ & v_y > 50), 10-frame confirm	sitting/bending
Loitering	Dwell time tiers	dwell $\geq 60/180/300$ s	grouping people
Abandoned Object	Owner-departure logic	stationary 30s + owner > 200px for 60s	owner nearby
Crowd Density	Grid + decaying heatmap	$\text{density} = \text{count} / (\text{H} \cdot \text{W}) \cdot 10^4$, EMA decay 0.95	perspective distortion
Fire/Smoke	HSV color + texture	weighted score, 60%-of-5-frame vote	fire-colored objects
Weapon	YOLOv8 + threat map	name match → CRITICAL/HIGH	base model not fine-tuned
Violence	Motion-stat fusion	weighted sum of 4 cues > sensitivity	needs ≥ 2 people
PPE	HSV color in body regions	color ratio > 15% per region	non-colored gear
ANPR	Haar/contour + EasyOCR	4-corner + aspect 2–5, OCR conf	plate angle/blur
Analytics	Stats + OLS regression	slope sign → trend	small samples

8. Flask Web Framework — Full Explanation

8.1 What is Flask?

Flask is a lightweight **Python web framework** ("micro-framework"). It lets you build web applications and REST APIs in Python. It is called "micro" because it ships only the essentials (routing, request/response, templating) and lets you add the rest via extensions. In this project Flask powers the **operator dashboard** — the web page where a guard watches the live feed and alerts.

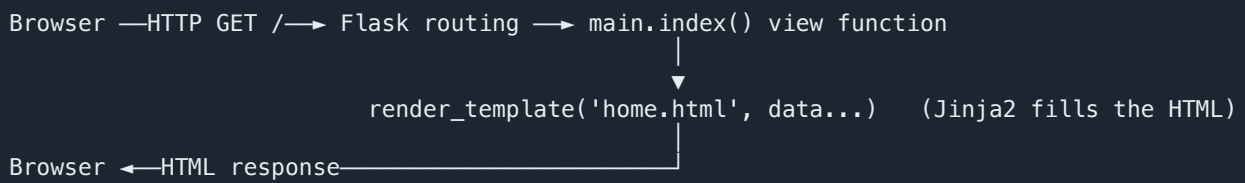
Core Flask concepts used here:

Concept	What it is	Used in this project
App / App Factory	The central Flask application object	<code>create_app()</code> builds and configures it
Route	A URL mapped to a Python function	<code>@main.route('/') →</code> home page
View function	The function that handles a request and returns a response	returns HTML or JSON
Blueprint	A group of related routes (modular)	<code>main</code> , <code>auth</code> , <code>api</code> blueprints
Template	An HTML file rendered with data (Jinja2)	<code>render_template('dashboard.html')</code>
Request	Incoming data (form, JSON, query args)	<code>request.get_json()</code> , <code>request.form</code>
Response / jsonify	What the server sends back	<code>jsonify({...})</code> for the API
Session	Per-user server-side state	login state in <code>auth_routes.py</code>
Static files	CSS/JS/images served as-is	<code>static/</code> folder

8.2 Why Flask (not Django) for this project?

- **Lightweight & fast to start** — ideal for a single-purpose dashboard.
- **Easy to embed** — the dashboard runs inside the surveillance process in a thread.
- **Flexible** — no imposed structure, easy to add a `/video_feed` streaming endpoint.
- **Great extension ecosystem** — `Flask-SocketIO` for real-time push, `Flask-Login` for auth.

8.3 How a request is served (the Flask request cycle)



For the **live video** it is different — the `/video_feed` route returns a **multipart MJPEG stream** (`Response(generate_frames(), mimetype='multipart/x-mixed-replace; boundary=frame')`). The generator yields JPEG-encoded frames continuously, so the `<img>` tag in the browser shows live video.

For **alerts**, instead of the browser polling, the server **pushes** via Socket.IO (WebSockets).

9. Main Flask Files and What Each Does

The dashboard lives in `src/dashboard/`. Here is every important file and its job.

`src/dashboard/__init__.py` — the Application Factory

- Creates the global `socketio = SocketIO()` instance.
- Defines `create_app()`, the **application factory** that:
- builds the `Flask(__name__)` app,
- sets config (`SECRET_KEY`, 16 MB upload cap, template auto-reload),
- initializes Socket.IO with CORS,
- **registers the three blueprints**: `main`, `auth`, and `api` (the API under the `/api` prefix).
- *Why a factory?* It avoids global state, makes testing easier, and lets you create multiple app instances with different configs.

`src/dashboard/app.py` — the Entry Point

- Imports `create_app()` and creates `app`.
- The `if __name__ == '__main__':` block runs the server on `127.0.0.1:<PORT>` (localhost is needed so the browser will grant camera permissions).

`src/dashboard/routes/main_routes.py` — Main Pages + Video Pipeline (the biggest file)

This is the **heart of the dashboard**. It defines the `main` blueprint and:

- **Page routes**: home, dashboard, feature-detail pages.
- `/video_feed` + `generate_frames()`: the core loop that reads frames, runs detection, tracking, facial recognition and all advanced detectors, draws overlays, encodes JPEG, and yields the MJPEG stream. This is where the **frame life-cycle** (Section 4) actually executes for the browser feed.
- **Component initialization** (`initialize_components()`): lazily

creates the detector, tracker, analyzer, and all advanced feature objects (configurable via env vars `YOLO_MODEL_SIZE`, `YOLO_IMGSZ`, `YOLO_CONF`, `YOLO_CLASSES`, `ENABLE_PPE_DETECTION`, ...). -

Camera management: `enumerate_cameras()`, `switch_camera()`. - **Daily stat counters** with anti-double-counting (counts a new object only when the tracker mints a new ID; counts a recognized face at most once per 10 s per name). - **Alert handling:** `StandardizedAlert.create_alert()` (throttling), `save_alert_to_db()` (persist + Socket.IO broadcast).

`src/dashboard/routes/auth_routes.py` — Authentication

- Defines the `auth` blueprint with `/login` and `/logout`.
- A `login_required` **decorator** protects routes (redirects to login if `'user'` not in session).
- Uses Flask **session** to remember the logged-in user.
- *Viva note:* the demo stores a plaintext password (`admin/password123`) — explicitly flagged as "never use in production"; the production fix is hashed passwords + a real user database.

`src/dashboard/routes/api_routes.py` — REST API (JSON)

- Defines the `api` blueprint, mounted at `/api`.
- Returns **JSON** (via `jsonify`) for the front-end's AJAX/fetch calls. Examples:
- `/api/health` — service health check.
- `/api/analytics/dashboard_data`, `/api/analytics/live_stats` — live counters & stats.
- `/api/facial_recognition/...` — enroll, list, remove, label captured faces, export/optimize/clear DB.
- `/api/behavior_analysis/...` — rules, events, patterns, toggle.
- `/api/person_reid/status`, `/api/system/config`.
- This is the **RESTful layer** that separates data (JSON) from presentation (HTML).

`src/dashboard/routes/feature_data.py` — Static Feature Content

- A big Python dictionary (`FEATURES_DATA`) describing each AI feature (title, metrics, pipeline, algorithm steps, code examples) used to render the **feature-detail pages**.
- *Viva note:* this is **documentation/marketing content**, not the live model — some descriptions are aspirational (see Section 5.2).

`src/dashboard/templates/*.html` — Jinja2 Templates (the View)

- `base.html` — shared layout (header, nav, styling) that other pages **extend**.
- `home.html` / `index.html` — landing & main dashboard pages.
- `dashboard.html` — the operator console (video feed, live stats, alert feed).
- `feature_detail.html` — renders one feature from `FEATURES_DATA`.
- `login.html` — login form.

- Jinja2 lets HTML include Python data with `{{ variable }}` and logic with `{% for %}` / `{% if %}`.

`src/dashboard/static/` — Static Assets

- Images/CSS/JS served directly (e.g., the system banner image).

How the pieces connect (one diagram)

```
app.py → create_app() — register main (pages + /video_feed + generate_frames)
                      — register auth (/login /logout, session)
                      — register api (/api/* JSON endpoints)
                      |
templates/*.html ← render_template
                      |
socketio.emit('new_alert') → browser (live)
                      |
static/* ← CSS/JS/img
```

10. Real-Time Communication (Socket.IO)

Normal HTTP is **request→response** (the browser must ask). For **instant alerts**, the project uses **Flask-SocketIO** (WebSockets), which keeps a persistent connection so the **server can push** to the browser the moment an event happens:

```
socketio.emit('new_alert', alert)    # server side
socket.on('new_alert', cb)          # browser side (JS)
```

- The server binds `socketio.run(app, host='127.0.0.1', port=8082)`.
- `cors_allowed_origins="*"` is set (convenient for dev; should be restricted in production).
- Benefit: **zero-latency alerts** without the browser polling every second.

11. Databases & Persistence

The project uses **SQLite** (a serverless, file-based SQL database) — one file per subsystem for clean separation. SQLite is ideal here: zero-config, embedded, perfect for a single-node app.

Database file	Stores
<code>alerts.db</code>	All generated alerts (id, type, message, priority, timestamp, JSON data)
<code>analytics.db</code>	Metrics, activity patterns, insights, benchmarks
<code>face_database.db</code>	Known faces, detections, unknown faces, captured-face gallery
<code>behavior_analysis.db</code>	Behavior events, pose data, crowd events
<code>person_reid.db</code>	Person gallery, tracks, cross-camera matches, cameras
<code>fall_detection.db</code> , <code>loitering_detection.db</code> , <code>abandoned_objects.db</code>	Per-event tables
<code>crowd_density.db</code> , <code>fire_smoke_detection.db</code>	Density snapshots / fire-smoke events
<code>weapon_detection.db</code> , <code>violence_detection.db</code>	Threat & violence events
<code>ppe_detection.db</code> , <code>license_plates.db</code>	PPE violations / plate reads + white/black lists

Binary artifacts: `face_encodings.pkl` (face embeddings), `person_features.pkl` (Re-ID gallery) are **pickled** for fast load. Face embeddings are tagged with the backend so they aren't reused across incompatible backends.

12. Performance Optimizations

Technique	File	Effect
Frame skipping (<code>FRAME_SKIP=3</code>)	<code>main_routes.py</code>	~3x fewer inferences; cache reused between
Threaded capture	<code>video_stream.py</code>	Camera I/O overlaps processing
GPU/MPS auto-select + FP16	<code>enhanced_detector.py</code>	Faster inference
Adaptive <code>imgsz</code> (960→640 on CPU)	<code>enhanced_detector.py</code>	Keeps FPS usable on CPU
Class allow-list	<code>main_routes.py</code>	Less postprocessing, fewer false labels
Alert throttling + dedupe	alert managers	Less DB/UI load, less noise
Bounded histories (last 1000 alerts)	alert managers	Bounded memory
EMA latency smoothing	<code>facial_recognition.py</code>	Stable telemetry
Gallery LRU eviction (Re-ID)	<code>person_reid.py</code>	Bounded memory

13. Key Mathematics Cheat-Sheet

Concept	Formula	Used in
Centroid	$((x_1+x_2)/2, (y_1+y_2)/2)$	tracking, all heuristics
Euclidean distance	$\sqrt{(\Delta x^2 + \Delta y^2)}$	tracking, fall, loiter, abandoned, violence
IoU	$\text{area}(A \cap B) / \text{area}(A \cup B)$	NMS, tracking, evaluation
Detection score	$P(\text{obj}) \cdot \max_c P(\text{class} \setminus \{\text{obj}\})$	YOLO
L2 norm	$\sqrt{(\sum v_i^2)}$	embedding normalization
Cosine similarity	$(a \cdot b) / (\ a\ \ b\)$	face & Re-ID matching
Face distance match	$\ a - b\ \leq 0.6$	dlib faces
Laplacian variance	$\text{Var}(\nabla^2 I)$	face sharpness / quality
LBP code	$\sum s(I_p - I_c) \cdot 2^p$	Re-ID texture
Density	$\text{count} / (H \cdot W) \cdot 10^4$	crowd
EMA / leaky integrator	$S \leftarrow \alpha \cdot S + (1 - \alpha) \cdot x$	heatmap decay, latency
OLS slope	$(n \sum xy - \sum x \sum y) / (n \sum x^2 - (\sum x)^2)$	analytics trend/prediction
Precision / Recall / F1	$TP / (TP + FP)$, $TP / (TP + FN)$, $2PR / (P + R)$	evaluation
Average Precision	$\int_0^1 P(R) \, dR$	mAP
Point-in-polygon	ray-casting / <code>pointPolygonTest</code>	restricted area / zones

14. Likely Viva Questions & Model Answers

Q1. Why YOLO and not Faster R-CNN? YOLO is single-stage — one forward pass predicts all boxes → real-time (30+ FPS). Faster R-CNN is two-stage (region proposal + classification) → more accurate but too slow for live video.

Q2. What is IoU and where is it used? Intersection-over-Union = overlap area ÷ union area of two boxes (0–1). Used in NMS (remove duplicate detections), in the tracker (associate boxes across frames), and in evaluation (a TP needs IoU ≥ threshold).

Q3. How does the tracker keep the same ID for a person? It builds a cost matrix from IoU and centroid distance, matches greedily (cheapest pair first), keeps an object alive for up to 30 missed frames, and stores a trajectory. (Upgrade: Deep SORT with Kalman + Hungarian for robustness.)

Q4. How does face recognition decide "known" vs "unknown"? Each face → 128-D embedding (SFace/dlib). Compare to stored embeddings: with SFace use **cosine similarity** (match if ≥ 0.363); with dlib use **Euclidean distance** (match if ≤ 0.6). Best match above threshold = that identity, else "Unknown".

Q5. What ML metrics evaluate the model and what do they mean? Precision (few false alarms), Recall (few misses), F1 (balance), mAP@0.5 and mAP@0.5:0.95 (mean average precision = area under PR curve, averaged over classes/IoU). Plus speed: FPS and latency.

Q6. What causes false positives/negatives and how do you reduce them? FPs: heuristic detectors (color), low thresholds → reduce with multi-frame verification & confidence gating. FNs: small/occluded objects, high thresholds → reduce with higher resolution / lower threshold (used for weapons where missing is unacceptable).

Q7. How do you prevent overfitting? Data augmentation, weight decay, dropout, early stopping, and a held-out validation set.

Q8. What is Flask and why use it here? A lightweight Python web framework. Used to build the operator dashboard: routes serve pages and a REST API, templates render HTML, Socket.IO pushes live alerts, and a streaming route delivers the MJPEG video feed.

Q9. Explain the main Flask files. `__init__.py` (app factory + blueprint registration), `app.py` (entry point), `routes/main_routes.py` (pages + video pipeline), `auth_routes.py` (login/session), `api_routes.py` (JSON API), `feature_data.py` (feature content), `templates/*.html` (Jinja2 views), `static/` (assets).

Q10. How is the video shown live in the browser? A `/video_feed` route returns a multipart MJPEG stream; `generate_frames()` continuously yields JPEG-encoded annotated frames, displayed in an `<img>` tag.

Q11. Why SQLite and not MySQL/PostgreSQL? Serverless, file-based, zero-config, embedded — perfect for a single-node app. For multi-node / high-concurrency you would move to a client-server DB.

Q12. How are alerts kept from spamming the operator? Per-type **throttling** (minimum seconds between same alert), **dedupe** of near-identical recent events, and **priority** classification (low/medium/high) so critical events stand out.

Q13. Is the system real-time? What makes it fast? Yes (~30 FPS target). Frame skipping (every 3rd frame), threaded capture, GPU/MPS + FP16, adaptive input resolution, and class filtering.

Q14. What are the main limitations? Several detectors are heuristic (color/geometry) and would benefit from trained CNNs; tracker has no motion model; Re-ID uses hand-crafted features; pixel thresholds aren't camera-calibrated. (See §15.)

15. Honest Limitations (and How to Defend Them)

Examiners respect candidates who know their system's weaknesses **and** the fix.

1. **Heuristic detectors** (fire/smoke, weapon, violence, PPE) use color/geometry/motion stats → false positives. **Fix:** fine-tuned CNNs (e.g., dedicated weapon/violence datasets). The code is structured so a trained `.pt` can be dropped in (`models/custom/`, `trainer.py`).
 2. **Greedy tracker** without motion prediction → ID switches on occlusion. **Fix:** Deep SORT (Kalman filter + Hungarian assignment + deep appearance features).
 3. **Person Re-ID** uses color-histogram + LBP, not a trained embedding → weak across lighting/pose. **Fix:** OSNet/ResNet ReID network.
 4. **Pixel-based thresholds** (speed, distance) depend on camera placement. **Fix:** camera calibration / homography to convert to metric units.
 5. **Security:** hard-coded demo credentials, secret-key fallback, open CORS. **Fix:** hashed passwords + real user DB, env-only secrets, restricted CORS, auth on all API routes.
 6. **No automated tests** for the analytics math/detectors. **Fix:** unit tests on formulas/thresholds.
 7. **Single camera** in the live loop (multi-camera class exists but isn't the main path). **Fix:** wire `MultiCameraManager` into the pipeline.
-

16. Glossary of Terms

- **YOLO** — You Only Look Once; single-stage real-time object detector.
- **CNN** — Convolutional Neural Network; learns spatial image features.
- **Bounding box (bbox)** — rectangle `[x1,y1,x2,y2]` around an object.
- **Confidence** — model's certainty a detection is correct (0–1).
- **IoU** — Intersection over Union; box-overlap metric.
- **NMS** — Non-Maximum Suppression; removes duplicate overlapping boxes.
- **Embedding** — fixed-length numeric vector representing an input (e.g., 128-D face vector).
- **Cosine similarity** — angle-based vector similarity (1 = identical direction).
- **mAP** — mean Average Precision; standard detection accuracy metric.
- **Precision / Recall / F1** — correctness, completeness, and their balance.
- **FPS / Latency** — speed metrics (frames per second / delay per frame).
- **MediaPipe** — Google library for body-pose landmarks.
- **OCR** — Optical Character Recognition (reading text from images).
- **HSV** — Hue-Saturation-Value color space (good for color thresholding).
- **LBP** — Local Binary Pattern; classical texture descriptor.
- **Flask** — lightweight Python web framework.

- **Blueprint** — a modular group of Flask routes.
- **Jinja2** — Flask's HTML templating engine.
- **Socket.IO / WebSocket** — persistent connection for server→browser push.
- **REST API** — HTTP endpoints exchanging JSON.
- **SQLite** — embedded, file-based SQL database.
- **Session** — per-user server-side state (used for login).
- **MJPEG** — Motion JPEG; stream of JPEG frames used for the live video.
- **EMA** — Exponential Moving Average; smooths a time series.
- **Overfitting / Underfitting** — memorizing vs. being too simple.

Prepared for viva-voce defense of the Smart Surveillance System. Pair this with `report.md` for the full mathematical derivations of every module.
